IP1 - pitanja

1. SVM

a. SVM

- SVM (Support Vector Machine) je algoritam za klasifikaciju čiji je cilj da pronađe **hiperravan koja maksimalno razdvaja dve klase**.
- Za razliku od običnog linearnog klasifikatora, SVM ne traži samo razdvajanje klasa, već traži razdvajanje sa **najvećom marginom**.

- **Osnovna ideja:**
  - Imamo dve klase označene sa −1 i +1.
  - Tražimo hiperravan oblika: $w \cdot x + b = 0$ koja razdvaja klase tako da je **rastojanje između najbližih tačaka dve klase maksimalno**.
  - To rastojanje se zove **margina**.

- **Šta znači "maksimalna margina"?**
  - Margina je rastojanje između dve paralelne ravni koje dodiruju najbliže tačke iz svake klase.
  - Što je margina veća: model je stabilniji, manja je verovatnoća prenaučenosti, bolja je generalizacija.

Zato je cilj SVM-a da minimizuje normu vektora w (jer je margina obrnuto proporcionalna toj normi).

- **Support vektori**
  - Ne određuju sve tačke klasifikacionu ravan.
  - Samo one tačke koje leže na ivici margine (najbliže ravni) utiču na model.
  - Te tačke se zovu **support vectors**.

To znači:

- model zavisi samo od malog broja tačaka,
- ostale tačke praktično nemaju uticaj.

Šta ako podaci nisu savršeno razdvojivi?

U realnim podacima često ne postoji savršena separacija.

Zato se uvodi **soft margin**:

- dozvoljava se da neke tačke budu pogrešno klasifikovane,
- uvodi se parametar C koji balansira:
  - širinu margine
  - kaznu za grešku

Ako je C velik → stroga klasifikacija (manje grešaka, uža margina)
Ako je C mali → tolerantniji model (šira margina, više grešaka dozvoljeno)

b. SVM – Lagrandžovi množioci i dualni problem

SVM rešava optimizacioni problem:

- minimizuje normu vektora w
- uz ograničenje da su sve tačke pravilno klasifikovane (ili unutar dozvoljene greške kod soft margin)

To je problem optimizacije sa ograničenjima.

Zašto Lagrandžovi množioci?

Kada imamo optimizaciju sa ograničenjima, uvodimo Lagrandžove množioce.

Formira se Lagrandžova funkcija koja uključuje:

- ciljnu funkciju
- ograničenja pomnožena odgovarajućim množiocima (α_i)

Ključni rezultat

Nakon rešavanja dobijamo:

w se može izraziti kao linearna kombinacija trening tačaka.

To je izuzetno važno jer znači:

Model zavisi samo od podataka, i to samo od onih sa nenultim množiocima (support vectors).

Dualni problem

Umesto da direktno rešavamo problem po w i b (primalni problem), možemo ga preformulisati u **dualni problem**.

Dualni problem:

- zavisi samo od Lagrandžovih množioca α_i
- sadrži samo skalarne proizvode između podataka

Zašto je to važno?

Jer:

- broj dimenzija atributa više nije presudan,
- omogućava primenu kernela.

Support vektori u dualnoj formi

Ako je $\alpha_i > 0$ → tačka je support vektor.

Ostale tačke imaju $\alpha_i = 0$ i ne utiču na model.

Zato je SVM "sparse" model.

Zaključak (ispitno)

SVM se rešava pomoću Lagrandžovih množioca jer je to optimizacija sa ograničenjima. Prelaskom na dualni problem dobijamo formu koja zavisi samo od skalarnih proizvoda između podataka, što omogućava primenu kernela i efikasno rešavanje problema.

c. SVM - kerneli i Mercerova teorema

Problem nelinearne separacije

Ako podaci nisu linearno razdvojivi u originalnom prostoru, možemo ih preslikati u višu dimenziju gde postaju razdvojivi.

To preslikavanje označava se funkcijom $\phi(x)$.

Međutim:

- eksplicitno preslikavanje može biti skupo ili nemoguće.

Kernel trik

U dualnoj formi SVM-a pojavljuju se samo skalarni proizvodi između podataka.

Zato možemo:

umesto $\phi(x_i)\cdot\phi(x_j)$ koristiti direktno funkciju

$K(x_i, x_j)$

To je kernel funkcija.

Time implicitno radimo u višoj dimenziji bez eksplicitnog preslikavanja.

Najčešći kerneli

- Linearni
- Polinomni
- RBF (Gaussian)
- Sigmoidni

RBF je najčešće korišćen jer može modelovati kompleksne granice odlučivanja.

Mercerova teorema

Ne može bilo koja funkcija biti kernel.

Da bi funkcija bila validan kernel, mora:

- biti simetrična
- biti pozitivno semidefinitna

Mercerova teorema daje uslov pod kojim funkcija predstavlja skalarni proizvod u nekom prostoru osobina.

Drugim rečima:

Ako funkcija zadovoljava Mercerove uslove, postoji preslikavanje u višu dimenziju gde ta funkcija odgovara običnom skalaru.

Zaključak (ispitno)

Kerneli omogućavaju da SVM radi u implicitno višedimenzionalnom prostoru bez eksplicitnog preslikavanja podataka.
Mercerova teorema garantuje da kernel funkcija zaista predstavlja skalarni proizvod u nekom prostoru osobina i da je metoda matematički ispravna.

2. Klasifikacija

a. Klasifikacija k najbližih suseda (KNN)

- Potrebno:
  - Skup sacuvanih slogova
  - Metrika za izracunavanje rastojanja izmedju slogova
  - Vrednost k koja predstavlja broj najblizih suseda koje razmatramo
- Za klasifikaciju nepoznatih slogova:
  - Izracunati rastojanje do ostalih slogova za trening
  - Odrediti k najblizih suseda
  - Koristeci oznake klasa najblizih suseda odrediti klasu nepoznatog sloga(npr.uzeti klasu vecine)
- Izbor vrednosti k:
  - Ako je k jako malo, klasifikacija je osetljiva na sum
  - Ako je k jako veliko, susedi mogu da ukljuce tacke iz drugih klasa
- Kada se glasa za vecinsku oznaku klase, svaki sused ima podjednak uticaj->algoritam je osetljiv na izbor K; jedan nacin da se smanji uticaj K je da se doda tezina na uticaj kod svakog suseda na osnovu distance tako da blizi susedi imaju veci uticaj

b. Klasifikacija zasnovana na pravilima (karakteristike i algoritmi, direktne metode)

- Klasifikator zasnovan na pravilima koristi skup pravila “ako... onda...” da bi se slogovi klasifikovali
- Svako klasifikaciono pravilo u skupu pravila moze se predstaviti na sl. nacin: r_i : ($uslov_i$) -> y_i
  leva strana pravila-(pred)uslov, desna strana pravila-posledica; uslov je konjunkcija atributa, y_i je oznaka klase
- Primer: (tip krvi = topla ^ nosi jaja = da) -> ptica
- Mere kojima se izrazava kvalitet klasifikacionog pravila
  - Odziv pravila: procenat broja slogova koji zadovoljavaju levu stranu pravila (TP/TP+FN)
  - Preciznost pravila: procenat broja slogova koji zadovoljavaju desnu stranu pravila od slogova koji zadovoljavaju levu stranu pravila (TP/TP+FP)
- Ogranicenja skupa pravila:
  - Uzajamno iskljuciva pravila
    - Ne postoje dva pravila koja pokrivaju istu istancu
    - Svaki slog je pokriven najvise jednim pravilom
  - Pravila pokrivaju sve mogucnosti
    - Klasifikator poseduje potpuno pokrivanje ako sadrzi pravilo za sve moguce kombinacije vrednosti atributa
    - Svaki slog je pokriven bar jednim pravilom
  - Zajedno ova dva ogranicenja osiguravaju da svaka instanca bude pokrivena tacno jednim pravilom
- Direktne metode:

- Pravila se izvajaju direktno iz podataka
- Algoritmi ripper, CN2, 1R
- Sekvencijalno pokrivanje
  1)pocinje se od praznog skupa pravila
  2)izdvajaju se pravila za narednu klasu
  3)uklanjaju se slogovi za trening koji su pokriveni dodtnim pravilom(pozitivni slogovi)
  4)ponavljaju se koraci od 2) do dostizanja kriterijuma zaustavljanja
- 1R (one rule)
  - Za svaki atribbut u skupu podataka formira se jedno pravilo i zatim se bira pravilo sa najvecom preciznoscu
  - Radi se sa atributima koji imaju diskretne vrednosti, dok nepoznate vrednosti tretira kao izdvojene(pojedinacne) vrednosti u skupu vrednosti atributa
- CN2
  - Formira uredjen skup pravila (prema kvalitetu)
  - Koristi sekvencijalno pokrivanje i odredjuje naredno pravilo bez fiksiranja klase unapred
  - Posledica – pravila za razlicite klase su pomesana
  - Nedostajuce vrednosti zamenjuje najcescim(kategoricki atr.) odnosno srednjim(numericki atr.) vrednostima
- Ripper
  - Koristi sekvencijalno pokrivanje
  - Formira uredjen skup pravila
  - Unapred se fiksira klasa i odredi skup pravila za tu klasu
  - Na narednu klasu se prelazi tek kada se kompletira prethodna klasa
  - Redosled pravila unutar grupe koja odredjuje jednu klasu nije bitan , dok je redosled pravila u uredjenoj listi diktiran velicinom klase , odnosno redosledom odredjivanja pravila

c. Algoritmi klasifikacije zasnovani na pravilima odlučivanja

d. Osnovni pojmovi klasifikacije

e. Merenje performansi (accuracy, precision, recall, F1, mere osetljive na cenu)

f. Poređenje rezultata klasifikacije

3. Stabla odlucivanja

a. CART

b. C4.5

- Sve metode kreiranja stable pocinju od korenskog cvora koji predstavlja ceo dati skup podataka I rekurzivno dele podatke na manje podskupove testiranjem odredjenog atributa u svakom cvoru. Ovaj process se nastavlja dok svi
  podskupovi ne budu "cisti"
- Koristi entropiju kao meru I odnos informacione dobiti kao kriterijum podele
- Kriterijum zaustavljanja:
  - Broj razlicitih instance je ispod zadate granice
  - Svi podaci u cvoru pripadaju istoj klasi
- Koristi numericke atribute (diskretizacija, sortiranje radi najbolje podele)
- Za klasu u listu se bira najbrojnija klasa (metoda glasanja)
- Vrsi se naknadno potkresivanje drveta :
  - Smanjenje nivoa greske (koristi se posebni test podaci)
  - Pesimisticka procena greske (ukljuceni sus vi podaci)
  - Intervali poverenja (Bernulijeva slucajna promenljiva)

c. C5.0

- Manji skup pravila (u odnosu na C4.5) sa istom preciznoscu
- Poboljsanje zastovano na dodatnom podsticanju (boosting) – kombinovanje razlicitih klasifikatora radi povecanja preciznosti
- Smanjeno koriscenje memorije za 90%
- Radi dosta brze od C4.5
- Nizi nivo greske I manje drvo odlucivanja
- Radi sa tezinama atributa

d. Svi algoritmi stabla osim C4.5 i CART

- ID3 – entropija kao mera, informaciona dobit kao krierijum podele;
  - kriterijum zaustavljanja: 1)svi podaci u cvoru pripadaju istoj klasi 2) najveca informaciona dobi <=0
  - ne vrsi se potkresivanje
  - ne radi sa numerickim atributima I nedostajucim vrednostima
  - formira binarno drvo
  - razmatra samo jedan atribut pri odlucivanju
- CHAID (Chi squared…)
  - Atribut klase moze biti samo **kategorciki,** dok ostali atributi mogu biti imenski, redi, kategoricki, neprekidni(neprekidni se transformisu u redne)
  - <u>Uparivanje</u> – neprekidni atributi se uparuju kao bi se od njih kreirale kategorije (za svaki atribut se odredjuje najslicniji par vrednosti)
  - <u>Podela</u> – za podelu se bira atribut sa najmanjom p-vrednoscu statistickog testa ; proverava se da li je p-vrendost veca od praga I ako jeste par se uparuje u jednu slozenu kategoriju I tgrazi se sledeci par vrednosti
  - <u>Zaustavljanje</u> – proces se zavrsava kada nema vise slozenih parova
- <u>QUEST</u>

    - Podrzava univarijantne(na osnovu jednog atributa) I linearne (na osnovu vise atributa) kombinacije podela vrednosti u cvoru
    - Veza izmedju atributa klase I atributa pri podeli:
        - Anova F—test ili Levane-ov test za redne I neprekidne atribute
        - X^2 za kategoricke atribute
    - Post-potkresivanje drveta
    - Atributi klasa kategoricki, ostali atributi imenski, redni, kategoricki, neprekidni
    - Binarna podela
- SLIQ- Supervised learning in QUEST
    - Varijanta QUEST-a razvijena u IBM-u za klasifikaciju velikih trening podataka
    - Radi efikasno sa podacima koji ne mogu da stanu u memoriju racunara
    - Nije zastovana na prethodno opisanom modelu- koristi se tehnika rasta drveta prvo u sirinu
    - Tehnika presortiranja u fazi rasta drveta
    - Koristi vertikalni format podataka – na pocetku se svi podaci sortiraju i smeste u listu
    - Podela na osnovu Gini indeksa
    - Koristi listu klasa koja cuva oznake klasa svake instance
    - Velicina liste klasa je direktno proporcionalna broju ulaznih slogova
    - Kategoricki i numericki atributi
    - Potkresivanje
- SPRINT
    - Modifikacija SLIQ-a koja uklanja memorijska ogranicenja
    - Oznake klasa pridruzuje identifikatorima instanci
    - Kategoricki i numericki atributi
    - Implementacija za serijsko i paralelno izvrsavanje

4. Ansambl metode

a. Ansambli

- Ansambli su metode mašinskog učenja kod kojih se **više modela kombinuje** da bi se dobio jedan konačni model sa boljim performansama.
- Osnovna ideja je: Umesto da se oslanjamo na jedan klasifikator, kombinuju se rezultati više klasifikatora.
- Time se postiže:
    - veća stabilnost,
    - bolja generalizacija,
    - smanjenje greške.

Zašto ansambli rade bolje?

Jedan model može:

- biti osetljiv na šum,
- biti nestabilan (npr. stabla odlučivanja),
- imati visok bias ili varijansu.

Kombinovanjem više modela:

- smanjuje se varijansa,
- ili se smanjuje bias,
- ili se balansiraju oba efekta.

Glavne vrste ansambala (kako su u PDF-u)

1 Bagging (Bootstrap Aggregating)

Ideja:

- iz trening skupa se više puta uzima bootstrap uzorak (uz vraćanje),
- na svakom uzorku se trenira poseban model,
- konačna odluka se dobija glasanjem (majority vote).

Karakteristike:

- modeli se treniraju paralelno,
- smanjuje varijansu,
- posebno efikasan za nestabilne algoritme (npr. stabla).

Najpoznatiji primer: Random Forest.

---

2 Random Forest

Specijalna forma bagging-a.

Pored bootstrap uzorkovanja:

- pri svakoj podeli u stablu bira se slučajan podskup atributa.

To dodatno:

- smanjuje korelaciju između stabala,

- poboljšava generalizaciju.

---

3 Boosting

Za razliku od bagging-a:

- modeli se treniraju sekvencijalno,
- svaki sledeći model fokusira se na greške prethodnog.

Ideja:

- povećava težinu pogrešno klasifikovanih instanci,
- teži da smanji bias.

Primer: AdaBoost.

Prednosti ansambala

- povećavaju tačnost
- stabilizuju modele
- smanjuju overfitting (kod bagging-a)
- često daju najbolje rezultate u praksi

Mane

- manja interpretabilnost
- veća računska složenost
- teže objašnjivi modeli

b. Klasifikacija metodom ansambla

Osnovna procedura

1. Izabere se osnovni klasifikator (npr. stablo).
2. Generiše se više verzija modela.
3. Kombinuju se njihove odluke.

Kombinacija se najčešće radi:

- većinskim glasanjem,

- ponderisanim glasanjem,
- prosekom verovatnoća.

Bagging u klasifikaciji

Koraci:

1. Iz originalnog skupa formiraju se bootstrap uzorci.
2. Na svakom uzorku trenira se klasifikator.
3. Za novu instancu svi modeli daju predikciju.
4. Klasa sa najviše glasova je konačna odluka.

Efekat:

- smanjenje varijanse,
- stabilniji model.

Boosting u klasifikaciji

Kod boosting-a:

1. Svaka instanca ima težinu.
2. Nakon svakog modela povećavaju se težine pogrešno klasifikovanih instanci.
3. Sledeći model više "obraća pažnju" na te instance.
4. Konačna klasifikacija je ponderisana kombinacija svih modela.

Efekat:

- smanjuje bias,
- može postići vrlo visoku tačnost.

Kada koristiti koji pristup?

- Ako je osnovni model nestabilan → bagging je dobar izbor.
- Ako model ima visok bias → boosting može pomoći.
- Random Forest je često univerzalno dobro rešenje.

Povezanost sa greškom modela

Ansambli balansiraju:

- bias
- varijansu

Bagging smanjuje varijansu.
Boosting smanjuje bias.

6. Klasterovanje

a. K- sredina, K-means, Rad sa praznim klasterima

K-means je algoritam za particiono klasterovanje koji deli skup podataka na K klastera.

Cilj:

- minimizovati unutar-klastersku varijansu,
- odnosno da tačke unutar klastera budu što bliže centru (centroidu).

Osnovna procedura

1. Odabere se broj klastera K.
2. Inicijalno se izaberu K centara (slučajno).
3. Svaka tačka se dodeljuje najbližem centru.
4. Centri se ažuriraju kao srednja vrednost svih tačaka u klasteru.
5. Koraci 3 i 4 se ponavljaju dok se klasteri ne stabilizuju.

Karakteristike K-means

- radi sa numeričkim podacima,
- koristi Euklidsku udaljenost,
- zahteva unapred zadat broj klastera,
- osetljiv je na inicijalizaciju i outliere,
- zahteva normalizaciju ako atributi imaju različite skale.

Problem praznih klastera

Može se desiti da nakon dodele:

- neki klaster ostane bez tačaka.

To se dešava ako:

- je inicijalni centar loše postavljen,
- svi njegovi članovi pređu u druge klastere.

Kako se rešava problem praznih klastera?

Tipične strategije:

1 Ponovno postavljanje centra na slučajnu tačku
2 Postavljanje centra na tačku koja je najudaljenija od postojećih centara
3 Podela najvećeg klastera na dva

U praksi se često koristi bolja inicijalizacija (npr. K-means++), da bi se smanjila verovatnoća praznih klastera.

b. Hijerarhijsko klasterovanje, Algoritmi hijerarhijskog klasterovanja

Hijerarhijsko klasterovanje gradi hijerarhiju klastera, obično predstavljenu dendrogramom.

Postoje dve vrste:

---

1 Aglomerativno (bottom-up)

- Svaka tačka počinje kao poseban klaster.
- U svakom koraku spajaju se dva najbliža klastera.
- Proces traje dok ne ostane jedan klaster.

---

2 Divizivno (top-down)

- Počinje sa jednim klasterom.
- U svakom koraku deli se klaster na manje.

Aglomerativno je češće u praksi.

Kako se definiše rastojanje između klastera?

To zavisi od metode povezivanja.

Najvažnije metode:

Single linkage

Rastojanje između dva klastera je minimalna udaljenost između njihovih tačaka.

→ formira “lančane” klastere.

Complete linkage

Koristi maksimalnu udaljenost između tačaka iz dva klastera.

→ formira kompaktne klastere.

Average linkage

Koristi prosečnu udaljenost svih parova tačaka.

Ward metoda

Spaja klastere tako da minimalno poveća ukupnu varijansu unutar klastera.

Prednosti

- ne zahteva unapred zadat broj klastera,
- daje kompletnu hijerarhiju rešenja.

Mane

- računski zahtevno za velike skupove,
- teško se ispravlja pogrešna odluka (nema vraćanja unazad).

c. Klasterovanje zasnovano na gustini i mrežama, DBSCAN tip, Grid-based pristupi, Klasterovanje kategoričkih podataka, Klasterovanje po potprostorima

Gustinsko klasterovanje – DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

Ideja:

- klaster je oblast visoke gustine tačaka,
- oblasti male gustine su razdvajanje ili šum.

Parametri:

- ε (radijus okoline),
- minPts (minimalan broj tačaka u okolini).

Tačka je:

- jezgro ako ima ≥ minPts suseda,
- granična ako je blizu jezgra,
- šum ako nema dovoljno suseda.

Prednosti DBSCAN

- ne zahteva broj klastera unapred,
- otkriva proizvoljne oblike klastera,
- prirodno detektuje anomalije.

Mane

- osetljiv na izbor parametara,
- slabiji u podacima promenljive gustine.

---

2 Grid-based (mrežni pristupi)

Prostor se deli na mrežu ćelija.

Klasteri se formiraju spajanjem gustih ćelija.

Primeri:

- STING
- CLIQUE (kod potprostora)

Prednosti:

- veoma brzi za velike skupove,
- složenost zavisi od broja ćelija, ne broja tačaka.

---

3 Klasterovanje kategoričkih podataka

K-means ne radi sa kategorijskim atributima jer koristi srednju vrednost.

Zato se koriste:

- K-modes (koristi mod umesto srednje vrednosti),
- mere neslaganja umesto Euklidske udaljenosti.

Ideja:

- centroid se definiše kao najčešća vrednost atributa.

---

4 Klasterovanje po potprostorima

Problem:
Kod visokodimenzionalnih podataka klasteri se često pojavljuju samo u nekim dimenzijama.

Subspace clustering:

- traži klastere u podskupovima atributa,
- ne mora svaki klaster koristiti sve dimenzije.

Primer:

- CLIQUE (kombinacija grid-based i potprostornog pristupa).

Važno kod:

- tekstualnih podataka,
- bioinformatike,
- velikih dimenzionalnih skupova.

d. Procena kvaliteta klasterovanja

Pošto klasterovanje nema unapred zadate klase, procena je složenija.

Postoje tri tipa mera:

---

1 Interna evaluacija

Zasniva se na:

- kompaktnosti klastera,
- razdvojenosti klastera.

Primer:

- unutar-klasterska varijansa,
- razlika između klastera.

Cilj:
→ tačke u istom klasteru bliske,
→ klasteri međusobno udaljeni.

---

2 Eksterna evaluacija

Ako su poznate prave klase:

- poredi se klasterovanje sa stvarnom podelom,
- koriste se mere slične klasifikaciji.

---

3 Relativna evaluacija

Upoređuje različite modele (različite vrednosti K, različite metode).

Primer:

- metoda lakta (elbow method) za izbor K,
- stabilnost klastera.

7. Otkrivanje anomalija

a. Tehnike detekcije anomalija

- Anomalija (outlier) je objekat koji značajno odstupa od ostalih podataka.
- To može biti: greška merenja, retki ali validan događaj, prevara, bezbednosni incident, neuobičajeno ponašanje.

Zašto su anomalije važne?

- Detekcija anomalija koristi se u: finansijskim prevarama, medicini, detekciji upada, kontroli kvaliteta, nadzoru sistema.

---

1 Statističke metode

Zasnivaju se na pretpostavci o raspodeli podataka.

Ideja:

- modeluje se normalno ponašanje (npr. normalna raspodela),
- tačke koje značajno odstupaju od očekivane vrednosti smatraju se anomalijama.

Primer:

- z-score (koliko je vrednost udaljena od srednje vrednosti),
- IQR (međukvartilni raspon).

Prednost:

- jednostavne i brze.

Mana:

- zavise od pretpostavke o raspodeli.

---

2 Metode zasnovane na udaljenosti

Ideja:

- tačka je anomalija ako je daleko od drugih tačaka.

Ako objekat nema dovoljno bliskih suseda unutar određenog radijusa → potencijalna anomalija.

Ove metode su povezane sa:

- k-NN pristupom,
- analizom gustine suseda.

Prednost:

- ne zahteva pretpostavku o raspodeli.

Mana:

- osetljive na dimenzionalnost.

---

3 Gustinske metode

Ideja:

- normalne tačke su u gustim regionima,
- anomalije su u retkim regionima.

Najpoznatiji primer: DBSCAN.

Ako tačka nema dovoljan broj suseda unutar ε → označava se kao šum (anomalija).

Takođe: LOF (Local Outlier Factor)

LOF upoređuje gustinu tačke sa gustinom njenih suseda.
Ako je lokalna gustina znatno manja → tačka je anomalija.

Prednost:

- mogu otkriti anomalije u klasterovanim podacima.

---

4 Klaster zasnovane metode

Prvo se izvrši klasterovanje.

Zatim:

- mali klasteri,
- izolovane tačke,
- tačke daleko od centroida

smatraju se anomalijama.

---

5 Model-based metode

Pravi se model “normalnog ponašanja”.

Sve što model ne može dobro da objasni smatra se anomalijom.

Primer:

- neuronske mreže,
- probabilistički modeli.

Zaključak (ispitno)

Tehnike detekcije anomalija mogu biti statističke, zasnovane na udaljenosti, gustini, klasterima ili modelima.
Izbor metode zavisi od tipa podataka i problema.

b. Određivanje anomalija

1 Na osnovu udaljenosti

Ako je tačka:

- daleko od većine drugih tačaka,
- nema dovoljno bliskih suseda,

onda se proglašava anomalijom.

Ovo je često formalizovano kroz:

- minimalan broj suseda,
- maksimalno dozvoljeno rastojanje.

---

2 Na osnovu gustine

Ako je lokalna gustina oko tačke znatno manja od gustine u okolini:

→ tačka je anomalija.

Ovaj pristup je bolji jer:

- uzima u obzir lokalnu strukturu,
- razlikuje retke, ali legitimne klastere od pravih anomalija.

---

3 Na osnovu statističkog odstupanja

Ako vrednost:

- značajno odstupa od srednje vrednosti,
- izlazi iz očekivanog opsega,

onda se označava kao anomalija.

Tipično:

- više od nekoliko standardnih devijacija od proseka,
- izvan IQR opsega.

---

4 U kontekstu klasterovanja

Ako tačka:

- ne pripada nijednom klasteru,
- nalazi se između klastera,
- pripada vrlo malom klasteru,

onda se smatra anomalijom.

---

5 U realnim podacima

Važno je naglasiti:

Nisu sve anomalije greške.
Neke su najvrednije informacije u skupu podataka.

Zato je:

- potrebno kombinovati metode,
- koristiti domensko znanje,
- pažljivo podešavati parametre.

8. Problematični podaci

a. Neuravnotežene klase

Neuravnoteženost (class imbalance) postoji kada je broj instanci u jednoj klasi znatno veći nego u drugoj.

Primer:

- 95% podataka pripada klasi “normalno”
- 5% klasi “prevara”

Ovo je vrlo čest slučaj u:

- detekciji prevara,
- medicinskoj dijagnostici,
- detekciji anomalija,
- bezbednosnim sistemima.

Zašto je to problem?

Standardni klasifikatori teže da minimizuju ukupnu grešku (accuracy).

U neuravnoteženom skupu može se desiti da:

- model klasifikuje sve instance kao većinsku klasu,
- dobije visoku accuracy,
- ali potpuno ignoriše manjinsku klasu.

Primer:
Ako 95% podataka pripada klasi A, model koji uvek predviđa A ima 95% tačnosti — ali je beskoristan.

Problemi u praksi

- Accuracy nije dobra mera performansi.
- Model postaje pristrasan ka većinskoj klasi.
- Manjinska klasa (često najvažnija) ostaje loše prepoznata.

Kako se rešava problem neuravnoteženosti?

1 Promena mera evaluacije

Umesto accuracy koriste se:

- precision
- recall
- F1
- ROC kriva
- AUC

Posebno je važan recall za manjinsku klasu.

---

2 Rebalansiranje podataka

a) Oversampling

- povećava se broj instanci manjinske klase
- kopiranjem ili generisanjem novih instanci

b) Undersampling

- smanjuje se broj instanci većinske klase

---

3 Ponderisanje klasa

U algoritam se uvode različite težine:

- greška nad manjinskom klasom kažnjava se više.

Ovo se često koristi kod:

- SVM (parametar C po klasi),
- neuronskih mreža,
- logističke regresije.

---

4 Specijalizovani algoritmi

Neki algoritmi su otporniji:

- ansambli,
- boosting,
- Random Forest.

Boosting je često dobar jer se fokusira na pogrešno klasifikovane instance.

Zaključak (ispitni odgovor)

Neuravnotežene klase predstavljaju problem kada jedna klasa znatno dominira u broju instanci.
Standardne mere poput accuracy tada nisu pouzdane.
Problem se rešava promenom mera evaluacije, rebalansiranjem podataka ili ponderisanjem grešaka.

b. Nedostajuće vrednosti

Nedostajuće vrednosti nastaju kada za određeni atribut ne postoji podatak.

Razlozi:

- podatak nije prikupljen,
- korisnik odbio da odgovori,
- atribut nije primenljiv,
- greška u sistemu.

Zašto su problem?

Većina algoritama:

- ne može direktno da radi sa praznim vrednostima,
- ili daje lošije rezultate ako ih ima mnogo.

Takođe:

- mogu narušiti statističke mere,
- uticati na klasifikaciju i klasterovanje.

Tehnike rešavanja nedostajućih vrednosti

1 Brisanje instanci

Ako je broj takvih instanci mali:

- mogu se ukloniti iz skupa.

Mana:

- gubitak podataka.

---

2 Imputacija (zamena vrednosti)

Najčešće metode:

- zamena srednjom vrednošću (za numeričke),
- zamena najčešćom vrednošću (za kategorijske),
- zamena vrednošću iz iste klase.

---

3 Model-based imputacija

- predikcija nedostajuće vrednosti pomoću drugih atributa,
- korišćenje regresije ili klasifikatora.

---

4 Algoritmi koji direktno rade sa nedostajućim vrednostima

Neki algoritmi, kao što je C4.5:

- raspodeljuju instancu po granama proporcionalno,
- ili koriste alternativne podele.

Kada je problem ozbiljan?

Ako nedostajuće vrednosti nisu slučajne, već sistematske, mogu:

- uvesti pristrasnost,

- promeniti raspodelu podataka,
- uticati na pouzdanost modela.

9. Pravila pridruzivanja

a. Osnovni koncepti

- Pravila pridruživanja služe za otkrivanje skrivenih veza između atributa u podacima.
- Najčešće se koriste kod tzv. transakcionih podataka, gde svaki slog predstavlja skup stavki (npr. proizvodi u korpi).
- Osnovna ideja:
  - posmatramo koje se stvari često pojavljuju zajedno.
  - Pravilo ima oblik:
    X → Y
    što znači: ako se pojavi X, često se pojavljuje i Y.
- Bitno:
  - X i Y su skupovi atributa (stavki)
  - pravilo pokazuje asocijaciju, ne uzročnost
- Primer razumevanja:

  nije da X izaziva Y, već da se statistički često pojavljuju zajedno.

Pravila pridruživanja (kategorička i neprekidna)

Kategorička

- rade direktno sa diskretnim vrednostima
- npr: proizvod = mleko, hleb
- ovo je osnovni i najčešći slučaj

Neprekidna

- rade sa numeričkim vrednostima
- ali se u praksi prvo radi **diskretizacija**
  (npr. plata → niska / srednja / visoka)
- bez diskretizacije, pravila se ne mogu direktno primeniti

Apriori princip

- Ovo je ključna ideja koja omogućava efikasnost.

- Ako neki skup nije čest → nijedan njegov nadskup ne može biti čest.
- Primer:
    - ako {mleko, hleb} nije često
    - onda {mleko, hleb, jaja} sigurno nije često
- Značaj:
    - znacajno smanjuje broj kombinacija koje treba proveravati
    - izbacuje loše kandidate vrlo rano

Apriori algoritam

- Apriori algoritam pronalazi **česte skupove stavki**, a zatim iz njih pravi pravila.
- Radi iterativno (po nivoima):
    - **1. Korak:** pronađu se svi **pojedinačni elementi** koji su česti
    - **2. Korak:** od njih se formiraju **parovi kandidata**
    - **3. Filtriranje:** zadržavaju se samo oni parovi koji su česti
    - **4. Dalje širenje:** formiraju se trojke, četvorke itd.
- svaki put se primenjuje Apriori princip
- proces se završava kada više nema novih čestih skupova

MinApriori (minimalna podrska)

- To je prag koji određuje: koliko puta neki skup mora da se pojavi da bi bio „važan“
- Ako je prag visok:
    - manje skupova
    - brži algoritam
- Ako je nizak:
    - više obrazaca
    - ali i više „šuma“ (nebitnih pravila)
- balans je jako bitan

Kreiranje i podkresivanje pravila

- Kada imamo česte skupove:
- **Kreiranje**
    - svaki skup se deli na X i Y
    - formiraju se pravila X → Y
- **Podkresivanje (pruning)**
    - proverava se kvalitet pravila
    - odbacuju se:
        - slaba
        - nepouzdana
        - nebitna pravila
- cilj je da ostanu samo **korisna i jaka pravila**

Smanjenje kandidata

- Problem: broj mogućih kombinacija eksplodira.
- Rešenje:
  - koristi se Apriori princip
  - generišu se samo oni kandidati koji imaju šansu da budu česti
  - ne praviš sve kombinacije, nego samo „logične“

Smanjenje broja poređenja

- **Problem koji se rešava**
  - Kod Apriori algoritma imamo ogroman problem:
    - za svaki kandidat (skup stavki) treba proveriti **koliko puta se pojavljuje u bazi**
  - To znači:
    - prolazak kroz celu bazu podataka
    - za svaki slog proveravaš da li sadrži taj skup
    - Ako ima puno kandidata → broj poređenja postaje ogroman (eksplozija kombinacija)

- **Šta znači „smanjenje broja poređenja“?**
  - Cilj je:
    - **da ne proveravamo svaki kandidat nad svim podacima**
  - odnosno:
    - da smanjimo broj:
    - prolaza kroz bazu
    - proveravanja kombinacija

- **Kako se to postiže:**
  - **1. Smanjenje broja prolaza kroz bazu**
    - Bez optimizacije: za svaki nivo (1-elementi, 2-elementi, 3-elementi…) ide se ponovo kroz celu bazu
    - Optimizacija: pokušava se da se **u jednom prolazu obradi više kandidata** ili da se baza „filtrira“ pa se dalje radi sa manjim skupom
    - manje skeniranja = manje poređenja

  - **2. Pametno filtriranje podataka**
    - Ideja: ne koristiš celu bazu ako ne moraš
    - Primer logike: ako neki slog ne sadrži nijedan čest element nema smisla koristiti ga za dalje kombinacije
    - Zato se: uklanjaju „nebitni“ slogovi , radi se sa **redukovanom bazom**
    - manje podataka → manje poređenja

  - **3. Organizacija kandidata (heširanje / strukture)**

        - Umesto da za svaki slog proveravaš sve kandidate: koristi se struktura (npr. heš stablo)
        - Kako to pomaže: kandidati su raspoređeni po „granama“
        - kada gledaš slog:
            - ideš samo kroz relevantne grane
            - ignorišeš ostale
        - ne poredиš sa svim kandidatima, već samo sa delom njih

---

Heš stablo

- Specijalna struktura za ubrzanje algoritma.
- Funkcija:
    - čuva kandidate :
        - Kandidati su: svi mogući skupovi stavki koje proveravamo (npr. {hleb, mleko})
        - Heš stablo ih: ne čuva kao običnu listu već ih raspoređuje u **strukturu nalik drvetu**
        - To znači: svaki kandidat ima svoje mesto u stablu -> lakše ga je pronaći kasnije
        - omogućava brzo pretraživanje
- Kako pomaže:
    - kandidati se raspoređuju po „granama“
    - Ovo je ključna ideja.
    - Kandidati se dele u grupe pomoću **heš funkcije**.
        - Primer: imaš kandidata: {hleb, mleko}, {hleb, jaja} , {mleko, jaja}
        - Heš funkcija odlučuje: u koju „granu“ stabla ide svaki kandidat
        - Tako dobiješ nešto poput: grana 1 → kandidati koji počinju sa „hleb“ , grana 2 → kandidati koji počinju sa „mleko“
        - kandidati nisu svi na jednom mestu, već su raspoređeni
        - pri proveri se ne gleda sve, nego samo odgovarajuća grana
    - smanjuje broj operacija i ubrzava brojanje pojavljivanja

b. Mere interesantnosti

- Kada dobijemo pravila tipa $\mathbf{X} \rightarrow \mathbf{Y}$, sledeće pitanje je:
- **da li je to pravilo zaista korisno ili je slučajno?**
- Zato se uvode **mere interesantnosti** koje procenjuju kvalitet pravila.

Support (podrska)

- Support pokazuje: **koliko često se zajedno pojavljuju X i Y u podacima**
- Drugim rečima: koliki deo svih podataka sadrži i X i Y
- **Kako da razumeš intuitivno:** Ako pravilo važi samo za 2 od 10 000 slučajeva: nije zanimljivo; Ako važi za 3000 od 10 000: ima smisla analizirati ga
- **Zašto je važan**

    - eliminiše retke i nebitne obrasce
    - koristi se kao prvi filter (MinApriori)
- bez supporta bi dobio gomilu beskorisnih pravila

Confidence (pouzdanost)

- Confidence pokazuje: **koliko često se Y pojavljuje kada se pojavi X**
- Znači: gledamo samo slučajeve gde postoji X i pitamo: koliko puta tada postoji i Y
- **Intuitivno :** Ako 100 ljudi kupi hleb, a 80 njih kupi i mleko: confidence je visok
  Ako samo 10 njih kupi mleko: confidence je nizak
- **Važna napomena:** Confidence može da zavara, jer ne uzima u obzir koliko je Y čest sam po sebi -> zato se koristi još jedna mera

Lift

- Lift meri: **da li je pojava Y stvarno povezana sa X ili je slučajna**
- Odnosno: poredi stvarnu povezanost sa očekivanom (da su nezavisni)
- ako je lift ≈ 1
  → X i Y nemaju veze
- ako je lift > 1
  → X povećava verovatnoću Y
- ako je lift < 1
  → X smanjuje verovatnoću Y
- **Zašto je bitan**
    - uklanja „lažno dobra“ pravila
    - Primer: ako svi kupuju mleko, onda će pravilo X → mleko imati visok confidence, ali to ne znači da X ima veze sa mlekom
    - Lift to otkriva.

Osobine mera

- **1. Support**
    - favorizuje česte obrasce
    - ignoriše retke (i možda zanimljive) veze
- **2. Confidence**
    - jednostavan za razumevanje
    - ali može biti **obmanjujući**
    - jer ne uzima u obzir osnovnu učestalost Y
- **3. Lift**
    - uzima u obzir zavisnost između X i Y
    - daje realniju sliku povezanosti
    - zato je često najinformativnija mera

Simpsonov paradoks

- To je situacija kada: neka veza važi u celini podataka, ali kada se podaci podele na grupe → ta veza nestaje ili se obrne.
- **Intuitivni primer**
  - U celom skupu: izgleda da X utiče na Y
  - Ali kada podeliš podatke (npr. po polu, starosti…): vidiš da u svakoj grupi veza ne postoji ili je suprotna
- **Zašto je ovo problem**
  - pravila mogu izgledati dobra
  - ali su zapravo posledica **skrivenih faktora**
  - možeš doneti pogrešne zaključke
- **Poenta za pravila pridruživanja**
  - mere (support, confidence, lift) gledaju agregatne podatke
  - ne vide skrivene podele
  - zato treba biti oprezan pri interpretaciji pravila

c. Vertikalni algoritmi

- **Osnovna ideja (šta znači „vertikalni“)**
  - Do sada (klasični Apriori) razmišljamo ovako: svaki slog = lista stavki (horizontalni pristup)
  - Primer: T1: hleb, mleko ; T2: mleko, jaja
- Kod **vertikalnih algoritama** menja se pogled na podatke:
  - svaki atribut (stavka) čuva listu transakcija u kojima se pojavljuje
  - Primer: hleb → T1, T3; mleko → T1, T2 ; jaja → T2
- Dakle:
  - horizontalno = transakcija → stavke
  - vertikalno = stavka → transakcije
- **Zašto se uvodi vertikalni pristup**
  - Problem kod klasičnog Apriori: mora stalno da prolazi kroz celu bazu, mnogo poređenja, sporo za velike skupove podataka
- Vertikalni pristup rešava to:
  - **ne moraš da skeniraš bazu svaki put**
  - već: koristiš već pripremljene liste transakcija; računaš preseke tih lista

Vertikalni Apriori

- ovo sto pise gore
- **Prednosti**
  - mnogo brže za velike podatke
  - manje prolaza kroz bazu
  - efikasnije brojanje učestalosti
- **Mana**
  - mora da se čuvaju liste transakcija
  - može zauzeti više memorije

Kompaktno predstavljanje čestih skupova

- Problem: broj čestih skupova može biti ogroman
- Zato se uvodi ideja: **ne čuvamo sve skupove, već samo reprezentativne**
- **Cilj:** smanjiti broj skupova a da se informacija ne izgubi
- **Ideja:** neki skupovi su „višak“ jer se mogu dobiti iz drugih; zadržavaju se samo oni koji nose novu informaciju
- rezultat:
    - manji skup
    - lakša analiza
    - brži rad

Ostali vertikalni algoritmi

- Osim vertikalnog Apriori, postoje i drugi algoritmi koji koriste isti princip: rad sa listama transakcija umesto skeniranja baze
- **Zajednička ideja svih**
    - vertikalna reprezentacija podataka
    - korišćenje preseka skupova
    - smanjenje broja operacija
- **Zašto su uvedeni**
    - da budu efikasniji od klasičnog Apriori
    - posebno za velike baze podataka
- **Razlike među njima (uopšteno)**
    - način čuvanja podataka
    - način generisanja kandidata
    - optimizacije u memoriji i brzini

d. FP rast / FP stablo

- Osnovna ideja (zašto se uvodi FP-rast):
- Kod Apriori algoritma problem je:
    - generiše se ogroman broj kandidata
    - više puta se prolazi kroz bazu
    - algoritam postaje spor
- FP-rast je uveden da to izbegne
- Glavna ideja: bez generisanja kandidata, direktno pronalazi česte skupove

FP stablo

- FP stablo je struktura koja: **kompaktno predstavlja celu bazu podataka**

- **Kako se pravi FP stablo**
    - **1. Prvo se pronađu česte stavke**
        - iz baze se izbacuju retke stavke
        - ostaju samo one koje zadovoljavaju minimalni prag
    - **2. Sortiranje stavki**

        - stavke se sortiraju po učestalosti
        - najčešće idu prve
        - ovo je bitno jer omogućava sabijanje stabla

    - **3. Izgradnja stabla**
        - Za svaku transakciju:
        - uzimaš samo česte stavke
        - sortiraš ih
        - ubacuješ u stablo kao putanju
    - ako više transakcija ima isti početak → dele granu

- **Kako izgleda rezultat**
    - stablo ima:
        - čvorove (stavke)
        - brojače (koliko puta se pojavljuju)
    - zajednički delovi transakcija se „spajaju“

- **Zašto je važno**
    - smanjuje veličinu podataka
    - eliminiše ponavljanje
    - omogućava efikasno pronalaženje obrazaca

FP rast

- FP-rast koristi FP stablo da pronađe česte skupove.

- **Kako radi**
    - **1. Kreće od najmanje čestih stavki**
        - uzima jednu stavku
    - **2. Pravi uslovnu bazu**
        - gleda gde se ta stavka pojavljuje u stablu
        - izvlači odgovarajuće putanje
    - **3. Pravi uslovno FP stablo**
        - od tih podataka gradi novo stablo
    - **4. Rekurzivno ponavlja proces**
        - pronalazi kombinacije
        - sve dok ne iscrpi sve mogućnosti

- **Ključna razlika u odnosu na Apriori**
    - Apriori → generiše kandidate i proverava
    - FP-rast → koristi stablo i direktno pronalazi skupove
    - nema eksplozije kandidata

- **Prednosti FP-rasta**

- nema generisanja kandidata
- manje prolaza kroz bazu (obično 2)
- brži za velike skupove podataka
- kompaktnija reprezentacija

Alternativne metode određivanja čestih stavki

10. Sekvencijalni obrasci

a. Istraživanje sekvencijalnih obrazaca

Sekvencijalni podaci su podaci kod kojih je redosled elemenata važan.

Za razliku od pravila pridruživanja, gde redosled nije bitan, kod sekvencijalnih obrazaca je ključno:

-šta se pojavljuje i kojim redosledom.

Primeri:

- kupovine kroz vreme,
- klikovi na web stranici,
- DNK sekvence,
- medicinski događaji kroz vreme.

Šta je sekvencijalni obrazac?

Sekvencijalni obrazac je niz stavki koji se pojavljuje u velikom broju sekvenci.

Na primer:
Ako mnogi korisnici prvo kupe A, zatim B, pa C — to je sekvencijalni obrazac.

Cilj istraživanja sekvencijalnih obrazaca

Pronaći sekvence koje:

- se javljaju često,
- imaju smislen redosled,
- mogu se koristiti za predikciju ili analizu ponašanja.

Razlika u odnosu na Apriori

Apriori traži česte skupove stavki bez reda.

Sekvencijalno rudarenje:

- uzima u obzir vremenski ili logički redosled,
- problem je složeniji jer je broj mogućih sekvenci mnogo veći.

Gde se koristi?

- analiza ponašanja kupaca,
- predviđanje sledećeg događaja,
- biomedicina,
- analiza korisničkih sesija.

b. Definicija podniske

Niska (sekvenca) je uređen niz elemenata.

Primer:
(A, B, C, D)

Šta je podniska?

Sekvenca S1 je podniska sekvence S2 ako:

- elementi S1 postoje u S2,
- redosled je očuvan,
- ali ne moraju biti uzastopni.

Primer:

S2: (A, B, C, D, E)
S1: (A, C, E)

S1 je podniska S2 jer:

- elementi postoje,
- redosled je isti,

- ali nisu susedni.

Ključna osobina

Redosled mora biti očuvan.
To je osnovna razlika u odnosu na obične skupove.

c. Neprekidna podniska

Neprekidna podniska je specijalni slučaj podniske.

Ovde važi:

- elementi moraju biti uzastopni,
- ne sme biti preskakanja.

Primer:

S2: (A, B, C, D, E)

(C, D) → jeste neprekidna podniska
(A, C) → nije neprekidna podniska

Zašto je važna razlika?

Neprekidne podniske su restriktivnije.

Koriste se kada:

- je važan kontinuitet događaja,
- redosled mora biti neposredan (npr. greške u sistemu, DNK sekvence).

d. Pronalazak sekvencijalnih obrazaca sa ograničenjima

Problem:

Broj mogućih sekvenci brzo raste sa dužinom sekvence.

Zato se uvode ograničenja (constraints).

Tipična ograničenja

1 Ograničenje dužine

Minimalna ili maksimalna dužina sekvence.

2 Vremensko ograničenje

Elementi moraju nastupiti unutar određenog vremenskog intervala.

3 Ograničenje razmaka (gap constraint)

Dozvoljeni broj elemenata između stavki u sekvenci.

4 Ograničenje vrednosti

Sekvenca mora sadržati određeni element.

Zašto su ograničenja važna?

- smanjuju broj kandidata,
- ubrzavaju algoritam,
- povećavaju relevantnost rezultata,
- eliminišu trivijalne ili besmislene sekvence.

Veza sa Apriori principom

Slično kao kod čestih skupova, koristi se princip:

Ako sekvenca nije česta → nijedna njena nadsekvenca ne može biti česta.

Ovo omogućava efikasno pretraživanje.

e. Sekvencijalna pravila

Šta su sekvencijalna pravila?

Sekvencijalno pravilo ima oblik:

A → B

i znači:

Ako se pojavi sekvenca A, verovatno će se kasnije pojaviti sekvenca B.

Bitna razlika u odnosu na pravila pridruživanja:

- postoji vremenski redosled.

Primer

Ako korisnik kupi:

(A, B)

onda će verovatno kasnije kupiti:

(C)

Ovo je sekvencijalno pravilo.

Mere koje se koriste

Kao i kod pravila pridruživanja:

- podrška (koliko sekvenci sadrži obrazac),
- pouzdanost (koliko često B sledi A).

Razlika u odnosu na asocijativna pravila

Asocijativna pravila:

- redosled nije bitan.

Sekvencijalna pravila:

- redosled je ključan.

11. Retki i negativni obrasci(chatgpt)

a. Retki obrasci

- Retki obrasci su obrasci sa malom podrškom koji često imaju veliku praktičnu vrednost. Klasični algoritmi ih ne pronalaze jer filtriraju po minimalnoj podršci, pa je potrebno posebno prilagoditi pragove ili koristiti alternativne tehnike.
- Retki obrasci (rare patterns) su skupovi stavki ili pravila koja imaju **nisku podršku**, ali mogu biti veoma značajni.
- Kod klasičnog Apriori algoritma tražimo **česte skupove stavki** (frequent itemsets), tj. one čija je podrška iznad minimalnog praga (min-support).
- Retki obrasci su upravo suprotno: javljaju se retko, ali mogu nositi važnu informaciju.

Zašto su važni?

- U mnogim realnim problemima najzanimljiviji obrasci su upravo retki: medicinska dijagnoza (retke kombinacije simptoma), otkrivanje prevara, bezbednosni incidenti, retki ali rizični događaji.
- Ako koristimo klasičan min-support prag: retki, ali značajni obrasci biće odbačeni.

Zato je istraživanje retkih obrazaca poseban problem.

Problem sa retkim obrascima

- Ako snizimo min-support da bismo pronašli retke obrasce: broj kandidata eksplodira, računanje postaje neefikasno, javlja se veliki broj nevažnih kombinacija.
- Zato je istraživanje retkih obrazaca: računski zahtevno, osetljivo na šum.

b. Negativno korelisani obrasci

- Negativno korelisani obrasci su skupovi stavki ili pravila kod kojih se dve stavke zajedno javljaju **ređe nego što bi se očekivalo ako su nezavisne**.
- Prisustvo jedne stavke smanjuje verovatnoću pojave druge. To znači da između njih postoji negativna povezanost.

Primer intuitivno

- Ako kupovina proizvoda A gotovo nikada ne ide zajedno sa proizvodom B, iako su oba pojedinačno česta, onda su A i B negativno korelisani.
- Takvi obrasci mogu biti: konkurentski proizvodi, međusobno isključivi simptomi, alternativne opcije izbora.

Kako se razlikuju od klasičnih pravila?

- Klasična pravila pridruživanja traže: visoku podršku, visoku pouzdanost, često i lift > 1 (pozitivna korelacija).
- Negativno korelisani obrasci imaju: lift manji od 1, zajedničku pojavu manju od očekivane.

Zbog toga ih standardni algoritmi često zanemaruju.

Zašto su važni?

- Mogu otkriti: konkurenciju proizvoda, alternativne navike kupaca, suprotne obrasce ponašanja, potencijalne greške u podacima.

---

c. Tehnike istraživanja negativnih obrazaca

- Tehnike istraživanja negativnih obrazaca uključuju:
  - upotrebu mera interesantnosti koje detektuju negativnu korelaciju,
  - proširenje algoritama da generišu negativna pravila,
  - statističko testiranje nezavisnosti,
  - prilagođavanje praga podrške,
  - vertikalne i kompaktnije pristupe rudarenju.

Poenta je da standardni Apriori algoritam nije dovoljan, jer eliminiše retke i negativno povezane obrasce.

Problem

Klasični Apriori algoritam:

- zasniva se na principu anti-monotoničnosti,
- filtrira po minimalnoj podršci,
- favorizuje česte skupove.

Negativni obrasci su često:

- retki,
- sa malom podrškom,
- pa ih Apriori automatski odbacuje.

Zato su potrebne posebne tehnike.

---

1 Korišćenje mera interesantnosti

Umesto oslanjanja samo na podršku, koriste se dodatne mere:

- lift,
- leverage (moc, snaga),
- conviction (osuda),
- korelacione mere.

Negativna korelacija se identifikuje kada je lift < 1.

Dakle:

- ne tražimo samo česte skupove,
- već i statistički značajne devijacije od nezavisnosti.

---

2 Odvojeno rudarenje negativnih pravila

Moguće je generisati pravila oblika:

A → ¬B
¬A → B

To zahteva:

- proširenje skupa stavki negativnim literama,
- pažljivo filtriranje da se izbegne eksplozija kandidata.

---

3 Prilagođavanje praga podrške

Jedan pristup je:

- koristiti različite pragove podrške za različite stavke,
- omogućiti niži prag za potencijalno interesantne retke stavke.

---

4 Statističko testiranje

Negativna korelacija se može proveriti statističkim testovima:

- test nezavisnosti,
- hi-kvadrat test.

Ovim se filtriraju slučajne negativne korelacije.

---

5 Kompaktno predstavljanje i vertikalni pristup

Kod vertikalnog formata (TID liste):

- podrška se dobija presekom skupova,
- lakše je analizirati odnose između stavki,
- može se efikasnije detektovati odstupanje od očekivane učestalosti.

12. Vizualizacija podataka

a. Načini prikaza podataka

- histogrami
- kucice (box plot)
- scatter plot- atributi vrednosti odredjuju poziciju, dodatni atributi mogu da se prikazu bojom, oblikom ili velicinom markera. upotrebe: graficki prikazuju vezu izmedju 2 atributa
- seme sa konturama - korisne su kada se neprekidni atributi mere na prostornoj osnovi, dele ravan na regione sa slicnim vrednostima, konturne linije koje cine granice regiona povezuju tacke sa istim vrednostima, prikaz temperature, padavina, vazdusnog pritiska
- seme sa matricama(toplotne mape) -korisne su kada su objekti sortirani u klase, uobicajeno je da se atributi normalizuju da bi se izbegladominacija jednog atributa, toplotne mape koje sadrze slicnosti ili razlike mogu da budu korisne u predstavljanju odnosa medju atributima
- paralelne koordinate - prikaz vrednosti atributa visedimenzionih podataka, umesto upravnih osa koriste se paralelne, svaki objekat se predstavlja linijom tako sto se vrednosti njegovih

atributa nalaze na koord. osama(tacke) a linija spaja te tacke, uredjenje atributa je vazno za uocavanje grupisanja

- crtanje zvedza- slicno kao kod paralelnih koord. samo se ose granaju iz jedne tacke u centru, tacke objekta su spojene poligonalnom linijom
- chernoff-ova lica- svaki atribut se pridruzuje jednoj karakteristici lica, vrednost aributa odredjuje izgled i karakteristike lica, objekat je pojedinacno lice
- dijagrami voronoia-

b. Tipovi podataka i odgovarajuće vizualizacije

13. Pretprocesiranje (chatgpt)

a. Pretprocesiranje podataka

Pretprocesiranje podataka predstavlja skup postupaka kojima se sirovi podaci dovode u oblik pogodan za primenu algoritama za klasifikaciju, klasterovanje i ostale metode informacionog pretraživanja.Podaci u realnim sistemima su često:

- nepotpuni (nedostajuće vrednosti),
- nekonzistentni,
- različitih tipova i skala,
- visoke dimenzionalnosti,
- sa šumom i duplikatima.

Zato je pretprocesiranje ključno za kvalitet modela.

1 Izdvajanje karakteristika

Prvi korak je transformacija sirovih podataka (tekst, slike, sekvence) u numeričke ili kategorijske osobine koje algoritmi mogu da koriste.

Na primer:

- tekst se predstavlja kao vektor termina,
- sekvence (npr. DNK) se transformišu u skup binarnih ili numeričkih osobina.

---

2 Transformacija tipova podataka

Ova faza obuhvata:

Diskretizaciju

Pretvaranje neprekidnih atributa u kategorije.

Koraci:

- odabir broja intervala,
- podela opsega vrednosti,
- mapiranje vrednosti u kategorije.

Načini izbora intervala:

- jednake širine,
- jednaka učestalost (isti broj elemenata po intervalu),
- log-intervali.

Diskretizacija se često koristi kod pravila pridruživanja i stabala.

Binarizaciju

Kategorijski atribut sa n vrednosti pretvara se u n binarnih atributa (one-hot kodiranje).

---

3 Čišćenje podataka

Obuhvata:

Nedostajuće vrednosti

Mogu se:

- obrisati slogovi,

- proceniti (imputirati),
- tretirati posebnim metodama u zavisnosti od algoritma.

Nekorektni i nekonzistentni podaci

Otkrivaju se pomoću:

- domenskog znanja,
- logičkih ograničenja,
- analize odstupanja.

Duplikati

Najčešće nastaju pri integraciji više izvora podataka i uklanjaju se.

---

4 Skaliranje i normalizacija

Potrebno kada atributi imaju različite skale.

Min-max normalizacija

x′=x−minmax−minx' = \frac{x - min}{max - min}x′=max−minx−min

Mapira vrednosti u interval [0,1].

Standardizacija (z-score)

z=x−μσz = \frac{x - \mu}{\sigma}z=σx−μ

Dovodi podatke na srednju vrednost 0 i standardnu devijaciju 1.

Važno je posebno kod metoda zasnovanih na udaljenosti (k-NN, SVM, K-means).

---

5 Redukcija podataka

Cilj je smanjenje dimenzionalnosti i količine podataka bez velikog gubitka informacije.

Metode:

- Agregacija (spajanje atributa ili objekata),
- Uzorkovanje (sampling),
- Izbor karakteristika (eliminacija redundantnih i irelevantnih),
- PCA (rotacija koordinatnog sistema i zadržavanje glavnih komponenti),
- SVD i druge metode dimenzione redukcije.

PCA omogućava:

- smanjenje dimenzionalnosti,
- uklanjanje korelacije između atributa,
- očuvanje maksimalne varijanse.